

TRAINING REPORT
On
EMPLOYEE AND ROLE MANAGEMENT SYSTEM

At



Defence Research and Development Laboratory (DRDL)
Kanchanbagh, Hyderabad 500058

in partial fulfilment of the degree

BACHELOR OF TECHNOLOGY

In

Computer Science & Engineering (CSE)

Guru Nanak Institute of Technology
Ibrahimpattanam, Telangana 501506



Submitted By

Kondamalla Sruthen Kumar 22831A0595

Meka Rochani Reddy 22831A05B3

Under the guidance of

Shri B.V. Hari Krishna Nanda (SCIENTIST 'G')
Technology Director , DIT, DRDL

DECLARATION

We, **Kondamalla Sruchen Kumar and Meka Rochani Reddy**, are hereby declare that this report is being submitted in partial fulfilment of the requirements of my internship at **Defence Research and Development Laboratory (DRDL)**.

This report is an original work carried out by us during our internship and have been completed under the guidance of **Shri B.V. Hari Krishna Nanda, Scientist 'G'** and **Smt. GMV Lakshmi**. This internship was undertaken as a part of the academic curriculum of **Guru Nanak Institute of Technology, Ibrahimpatnam, Telangana-501506**.

Signature

Kondamalla Sruchen Kumar

Signature

Meka Rochani Reddy

Signature

Smt. G M V Lakshmi

Signature

B V HARI KRISHNA NANDA , SCIENTIST 'G'

Technology Director , DIT

ACKNOWLEDGEMENT

We extend our sincere gratitude to **Shri. B. V. Hari Krishna Nanda (Scientist ‘G’)** for providing us the invaluable opportunity to undergo one-month industrial training and for his expert guidance throughout the project.

We are profoundly thankful to our guide, **Smt. G. M. V. Lakshmi**, for her constant support and supervision during the training period and in the preparation of this report. Their insights were instrumental in helping us successfully complete our project on **Employee and Role Management System**.

We also wish to express our gratitude to all the faculty members of the Department of Computer Science and Engineering (CSE) for their encouragement and support during this training.

Finally, we would like to thank the library and all other staff members who provided access to essential resources that greatly aided our learning.

Signature
Kondamalla Sruchan Kumar
(22831A0595)

Signature
Meka Rochani Reddy
(22831A05B3)

Defence Research and Development Laboratory (DRDL), located in Hyderabad, is a premier laboratory engaged in the research and development of missile systems for the Indian Armed Forces. DRDL focuses on the design, development, and integration of advanced missile technologies to enhance national defence capabilities.



The laboratory works in key areas such as missile propulsion, guidance and control, avionics, structures, and system integration. DRDL is equipped with modern facilities and advanced infrastructure to support high-quality research, testing, and development activities.

DRDL also provides opportunities for students to gain practical exposure through internship programs, enabling them to understand professional research practices, teamwork, and discipline in a defence organisation.

HISTORY

The Defence Research and Development Laboratory (DRDL) was established in 1961 under the Defence Research and Development Organisation (DRDO) with the primary objective of developing indigenous missile systems for India's defence forces. Located at Kanchanbagh, Hyderabad, DRDL marked the beginning of India's journey towards self-reliance in missile technology during an era when the country was largely dependent on foreign defence equipment.

In its formative years, DRDL initiated several pivotal programmes such as Project Indigo, a surface-to-air missile project which, although not fully successful, laid the groundwork for more advanced initiatives. This was followed by Project Devil and Project Valiant in the 1970s, which contributed significantly to the development of short-range missile technologies and laid a strong foundation for future projects.

The laboratory's efforts culminated in the launch of the Integrated Guided Missile Development Programme (IGMDP) in the 1980s, a major milestone in India's missile development history. DRDL played a critical role in the successful design and development of key strategic missile systems under this programme, including Prithvi, Agni, Akash, and Nag.

Over the years, DRDL has evolved into a premier centre of excellence in missile technology, specialising in areas such as aerodynamics, propulsion, guidance and control, warhead technology, and missile systems integration. With state-of-the-art infrastructure for design, simulation, testing, and validation, DRDL supports end-to-end development of both tactical and strategic missile systems.

Today, DRDL continues to drive innovation by contributing to advanced and next-generation missile systems such as hypersonic weapons, interceptor missiles, and long-range strategic deterrents. In alignment with DRDO's vision of self-reliance in defence technologies, DRDL actively collaborates with other DRDO laboratories, academic institutions, and Indian industries. Its continued commitment to technological excellence plays a vital role in enhancing India's defence preparedness and securing its strategic autonomy.

ABSTRACT

In modern organizations, efficient management of employee information is essential for smooth administrative operations and decision-making. Traditional methods of maintaining employee records using manual documentation or spreadsheets are time-consuming, error-prone, and lack scalability. To overcome these limitations, the Employee Management System has been developed as a full-stack web-based application that digitizes and centralizes employee record management.

The system is designed using React.js and Ant Design for the frontend to provide an interactive and responsive user interface, while Spring Boot is used in the backend to implement RESTful APIs and business logic. Oracle Database serves as the data storage layer, ensuring structured and reliable management of employee, role, division, and directorate information. Spring Data JPA (Hibernate) is utilized for efficient object-relational mapping and database interaction.

The application enables administrators to perform essential CRUD (Create, Read, Update, Delete) operations on employee records in real time. It maintains proper organizational hierarchy by establishing relationships between employees, roles, divisions, and directorates. The system follows a three-tier architecture, ensuring separation of concerns, scalability, and maintainability.

By automating employee data management, the system reduces administrative workload, improves data accuracy, and ensures centralized access to information. The modular architecture allows future enhancements such as authentication, role-based access control, reporting, and analytics dashboards. Overall, the Employee Management System provides a reliable, scalable, and technology-driven solution for effective workforce management.

CONTENTS

S.NO	CONTENT	
	CERTIFICATE	
	DECLARATION	
	ACKNOWLEDGEMENT	
	HISTORY	
	ABSTRACT	
	TABLE OF CONTENT	
	LIST OF FIGURES	
	PART – A Orientation on Software Applications and Tools Used at DRDL	Duration
1	Understanding the Software Environment	1 month
2	Learning the Tools	2 months
3	Design of the Software System	1 month
4	Development of the Software System	1 month
5	Testing and Integration of the Software System	1 month
	PART-B Advertisement Management System	
1	CHAPTER 1 : INTRODUCTION	

	1.1	Background
	1.2	Objectives of the Project
	1.3	Problem Statement
	1.4	Advantages of Proposed System
	1.5	Scope of the Project
	1.6	Tools and Technologies
2	CHAPTER 2 : SYSTEM DESIGN & IMPLEMENTATION	
	2.1	System Architecture
	2.2	Module Description
	2.3	Implementation
	2.4	Workflow of the System
3	CHAPTER 3 : TESTING AND EVALUATION	
4	CHAPTER 4 : RESULTS	
	4.1	Output Snapshots
	4.2	Challenges Faced and Learning Outcomes
5	CHAPTER 5 : CONCLUSION & FUTURE ENHANCEMENT	

LIST OF FIGURES

Fig.NO	FIGURE
2.1.1	System Architecture
2.4.1	Frontend Code
2.4.2	Backend Code
4.1.1	Dashboard Page
4.1.2	Search Employee Page
4.1.3	View Employee Page
4.1.4	Update Employee Page
4.1.5	Role View Page
4.1.6	Manage Roles Page

PART A

ORIENTATION ON SOFTWARE APPLICATIONS AND TOOLS USED AT DRDL

Understanding the Software Environment

In this phase, the software development environment required for the Employee Management System was studied and configured. The project follows a full-stack architecture consisting of frontend, backend, and database components.

The frontend development environment was set up using **Node.js**, **React.js**, and **Vite**, while the backend environment was configured using **Java** and the **Spring Boot framework**. **Oracle SQL Developer** was used to create, manage, and maintain the relational database.

Development tools such as **Visual Studio Code** were used for frontend development, and **Spring Tool Suite (STS)** or **IntelliJ IDEA** was used for backend development. The system architecture, layered structure, and communication between frontend, backend, and database using RESTful APIs were studied in detail.

This phase provided a clear understanding of how the Presentation Layer (React), Application Layer (Spring Boot), and Data Layer (Oracle) interact to form a complete and structured web-based employee management application.

Learning the Tools

In this phase, the technologies and tools required for developing the Employee Management System were learned and practiced.

React.js was studied for developing the user interface and managing dynamic components. **Ant Design** was used to implement professional UI elements such as forms, tables, layouts, input fields, and dropdowns.

Spring Boot was learned for backend development, including:

- Creating RESTful APIs
- Implementing Controller, Service, and Repository layers

- Handling business logic
- Managing validation

Oracle SQL was used to design and manage database tables such as:

- EMPLOYEE
- ROLE
- DIVISION
- DIRECTORATE

Additionally:

- **Axios** was used for frontend-backend communication.
- **Spring Data JPA and Hibernate** were studied for database interaction and object-relational mapping.
- REST API development and JSON data exchange were practiced.

This phase ensured technical proficiency in full-stack development concepts required for the project.

Design of the Software System

In this phase, the overall design of the Employee Management System was created.

The system architecture was designed using a **three-tier architecture**, consisting of:

- **Presentation Layer (React Frontend)**
- **Business Logic Layer (Spring Boot Backend)**
- **Data Layer (Oracle Database)**

The database schema was designed with proper primary key and foreign key relationships between:

- Employee
- Role
- Division
- Directorate

UML diagrams such as:

- Use Case Diagram
- Class Diagram
- Activity Diagram
- Sequence Diagram

were designed to represent system functionality and structure.

The system modules were identified, including:

- Employee Management Module
- Role Management Module
- Division Management Module
- Directorate Management Module
- REST API Communication Module

The workflow of employee creation, updating, deletion, and retrieval was also designed during this phase.

Development of the Software System

In this phase, the Employee Management System was implemented using the selected technologies.

The frontend was developed using **React.js and Ant Design**, and pages such as:

- Home Page
- View Employees
- Update Employee
- View Roles

were created.

The backend was implemented using **Spring Boot**, where REST APIs were developed to handle employee operations such as create, view, update, and delete. The layered structure (Controller, Service, Repository) was implemented to maintain clean architecture.

The Oracle database was created, and tables were defined with appropriate

constraints and relationships to maintain data integrity.

All system components were integrated to ensure proper communication between frontend, backend, and database.

Testing and Integration of the Software System

In this phase, the system was tested to ensure proper functionality and performance.

Functional testing was performed to verify:

- Employee creation
- Viewing employee records
- Updating employee information
- Deleting employee records

API testing was conducted to ensure proper communication between the React frontend and Spring Boot backend.

Database testing was performed to verify correct storage, retrieval, updating, and deletion of employee records in the Oracle database.

All modules were integrated, and errors were identified and resolved. The system was evaluated for performance, usability, reliability, and accuracy.

The testing process confirmed that the Employee Management System operates correctly and fulfills the intended project requirements.

PART B

ADVERTISEMENT MANAGEMENT SYSTEM

CHAPTER 1

INTRODUCTION

1.1 BACKGROUND

In today's digital era, organizations increasingly rely on efficient information systems to manage their workforce effectively. As companies grow in size and complexity, managing employee records manually becomes inefficient, error-prone, and time-consuming. Traditional paper-based or spreadsheet-based systems lack scalability, security, and real-time accessibility, leading to data inconsistencies and administrative delays.

Employee data such as personal details, department allocation, roles, and organizational hierarchy must be maintained accurately for smooth operational functioning. Without a centralized system, maintaining records across multiple departments becomes difficult, often resulting in duplication of data, loss of information, and reduced productivity.

To address these challenges, the Employee Management System has been developed as a full-stack web application that digitizes employee record management. The system provides a centralized platform where administrators can create, view, update, and delete employee records efficiently.

The application is built using modern web technologies:

- Frontend: React.js with Ant Design for responsive user interface
- Backend: Spring Boot for RESTful API development
- Database: Oracle Database for structured data storage
- ORM: Spring Data JPA (Hibernate) for database interaction

This system ensures data integrity, secure access, real-time updates, and improved administrative efficiency. By integrating frontend and backend technologies in a structured architecture, the Employee Management System provides a scalable and maintainable solution for workforce management.

The project demonstrates the practical implementation of full-stack development principles, including REST API integration, layered architecture, and relational database management.

1.2 OBJECTIVE OF THE PROJECT

The primary objective of the Employee Management System is to develop a centralized digital platform for managing employee records in an organized and efficient manner.

The specific objectives include:

1. Centralized Employee Record Management To maintain all employee details such as name, role, division, and directorate in a single secure database.
2. CRUD Operations Implementation To implement Create, Read, Update, and Delete operations using RESTful APIs.
3. Role and Division Management To manage hierarchical relationships between employees, roles, divisions, and directorates.
4. User-Friendly Interface To design an interactive and responsive UI using React and Ant Design.
5. Database Integration To integrate Oracle Database using Spring Data JPA for efficient data handling.
6. Full-Stack Integration To demonstrate seamless communication between frontend and backend through REST APIs.

Scalability and Maintainability To develop the system using modular architecture for future enhancements.

1.3 PROBLEM STATEMENT

The proposed system, Employee Management System, is designed to provide a centralized, secure, and efficient digital platform for managing employee-related information within an organization. The system replaces traditional manual methods of maintaining employee records with a structured web-based solution built using modern full-stack technologies.

The system enables administrators to perform complete employee lifecycle management through an intuitive user interface. It allows the creation, retrieval, updating, and deletion of employee records in real time. Each employee is associated with specific roles, divisions, and directorates, ensuring proper organizational hierarchy management.

The proposed system follows a three-tier architecture consisting of:

- Presentation Layer (Frontend): Developed using React.js and Ant Design, providing a responsive and user-friendly interface for interacting with employee data.

- Application Layer (Backend): Implemented using Spring Boot, which exposes RESTful APIs to handle business logic, validation, and request processing.
- Data Layer (Database): Oracle Database is used to store structured employee, division, directorate, and role information. Spring Data JPA is used for efficient object-relational mapping and database communication.

The system ensures seamless communication between frontend and backend using REST APIs. Data is exchanged in JSON format, enabling fast and efficient interaction between the client and server.

- Real-time data updates
- Data consistency and integrity
- Reduced administrative workload
- Structured organizational hierarchy
- Improved accuracy and reliability

The system is designed to be scalable and modular, allowing future enhancements such as authentication mechanisms, role-based access control, reporting modules, and dashboard analytics.

Overall, the proposed Employee Management System provides a robust, maintainable, and technology-driven approach to workforce management, ensuring improved productivity and streamlined administrative operations.

1.4 Advantages of Proposed System:

The proposed Employee Management System offers several advantages over traditional manual or semi-digital record management methods. The system enhances operational efficiency, data accuracy, and organizational transparency through structured and automated processes.

The key advantages are as follows:

1. Centralized Data Management

All employee-related information is stored in a centralized Oracle database, reducing data redundancy and ensuring consistency across the organization.

2. Efficient CRUD Operations

The system enables quick Create, Read, Update, and Delete operations through RESTful APIs, allowing administrators to manage records in real time without delays.

3. Improved Data Accuracy and Integrity

Validation mechanisms in the backend ensure that only correct and structured data is stored in the database, minimizing human errors.

4. **User-Friendly Interface**

The frontend developed using React and Ant Design provides an interactive and responsive user interface, making it easy for users to navigate and perform operations efficiently.

5. **Structured Organizational Hierarchy Management**

The system maintains proper relationships between employees, roles, divisions, and directorates, ensuring clear organizational structure management.

6. **Scalability and Modularity** The layered architecture (Frontend, Backend, Database) allows the system to be easily scalable and extendable for future enhancements such as authentication, reporting modules, and analytics dashboards.

7. **Reduced Administrative Workload** Automation of employee record management reduces paperwork and manual tracking, saving time and effort for administrative staff.

8. **Real-Time Data Access** Any updates made in the system are reflected immediately, ensuring that decision-makers always have access to up-to-date information.

9. **Secure Data Handling** The use of Spring Boot and JPA ensures structured database access, and the system can be enhanced with authentication and authorization mechanisms for improved security.

Full-Stack Integration Experience The system demonstrates seamless integration between React frontend, Spring Boot backend, and Oracle database, ensuring reliable and efficient communication across layers..

1.5 SCOPE OF THE PROJECT

The scope of the **Employee Management System** includes the design and development of a full-stack web-based application aimed at simplifying and digitizing employee record management within an organization. The system focuses on maintaining structured and centralized employee information through an integrated frontend, backend, and database architecture.

The project primarily covers the following functional areas:

- Creation and maintenance of employee records
- Management of employee roles

- Division and directorate management
- Updating and deletion of employee information
- Viewing employee details in structured tabular format
- Establishing relationships between employees, roles, divisions, and directorates
- Implementation of RESTful APIs for frontend-backend communication
- Integration with Oracle Database using Spring Data JPA

The system enables real-time interaction between the React-based frontend and the Spring Boot backend, ensuring smooth data exchange using JSON format. The use of Oracle Database ensures structured relational data storage with referential integrity.

The current scope of the project is limited to administrative management of employee data. It does not include advanced Human Resource functionalities such as:

- Payroll management
- Attendance tracking
- Leave management
- Performance evaluation
- Recruitment management

However, the architecture is modular and scalable, allowing future enhancements such as:

- User authentication and role-based access control
- Advanced search and filtering
- Report generation and data analytics dashboards
- Pagination and large-scale data handling
- Audit logging and activity monitoring
- Cloud deployment and containerization

1.6 TOOLS AND TECHNOLOGIES

The development of the Employee Management System was carried out

using modern full-stack development tools and technologies. These tools were selected to ensure scalability, performance, maintainability, and structured system design.

1. Frontend Development Tools

React.js: React.js is a JavaScript library used for building dynamic and interactive user interfaces. It enables component-based development and efficient rendering using the Virtual DOM.

Ant Design (AntD): Ant Design is a UI component library used to design responsive and professional user interfaces. It provides pre-built components such as tables, forms, buttons, and modals.

Vite: Vite is a modern frontend build tool used for fast development and optimized production builds.

Axios: Axios is used for making HTTP requests from the frontend to the backend REST APIs.

React Router DOM: Used for navigation and routing between different pages within the application.

2. Backend Development Tools

Spring Boot: Spring Boot is used to develop the RESTful backend services. It simplifies application configuration and provides embedded server support.

Spring Data JPA: Used for Object-Relational Mapping (ORM) and database interaction.

Hibernate: Hibernate is the ORM framework that handles mapping between Java objects and Oracle database tables.

Maven: Maven is used for project dependency management and build automation.

3. Database Tools

Oracle Database: Oracle Database is used as the relational database management system to store employee-related data securely and efficiently.

SQL Developer: Oracle SQL Developer is used for database creation, table

management, and executing SQL queries.

4. Development Environment

Visual Studio Code: Used for frontend development (React application).

IntelliJ IDEA / Eclipse: Used for backend development (Spring Boot application).

Postman: Used for testing REST APIs during backend development.

5. Version Control

Git: Used for version control and code management.

GitHub: Used for hosting the project repository and maintaining source code versions.

CHAPTER 2

SYSTEM DESIGN & IMPLEMENTATION

2.1 SYSTEM ARCHITECTURE

The Employee Management System follows a Three-Tier Architecture, which separates the system into three major layers: Presentation Layer, Application Layer, and Data Layer. This layered approach ensures modularity, scalability, maintainability, and efficient data management.

1. User Layer (Client Side)

At the top of the architecture, users interact with the system through a web browser such as Chrome, Edge, or Firefox. The user performs actions like:

- Viewing employee records
- Adding new employees
- Updating employee details
- Deleting employee information

The communication between the browser and frontend occurs through HTTP/HTTPS protocols, ensuring secure and structured data transfer.

2. Presentation Layer (Frontend – React.js + Ant Design)

The Presentation Layer is responsible for handling user interaction and displaying data.

It is developed using:

- React.js for building dynamic user interfaces
- Ant Design for professional UI components
- Vite as the development and build tool

Responsibilities:

- Display employee data in table format

- Provide forms for adding and updating employees
- Handle user inputs
- Send REST API requests to backend
- Receive JSON responses and update UI dynamically

When a user performs an action (like clicking “Add Employee”), the frontend sends a REST API request in JSON format to the backend.

3. Application Layer (Backend – Spring Boot)

The Application Layer contains the business logic of the system. It is implemented using Spring Boot, which handles request processing and database interaction.

This layer is internally divided into:

Controller Layer

- Handles HTTP requests
- Maps API endpoints
- Accepts and returns JSON data

Example endpoints:

- GET /employees
- POST /employees
- PUT /employees/{id}
- DELETE /employees/{id}

Service Layer

- Contains business logic
- Performs validations
- Processes employee updates

- Ensures proper relationships between employee, role, division, and directorate

Repository Layer (JPA)

- Uses Spring Data JPA
- Communicates with Oracle Database
- Performs CRUD operations

The backend communicates with the database using JDBC with Hibernate ORM, which converts Java objects into relational table records.

4. Data Layer (Oracle Database)

The Data Layer stores all structured organizational data.

Main tables include:

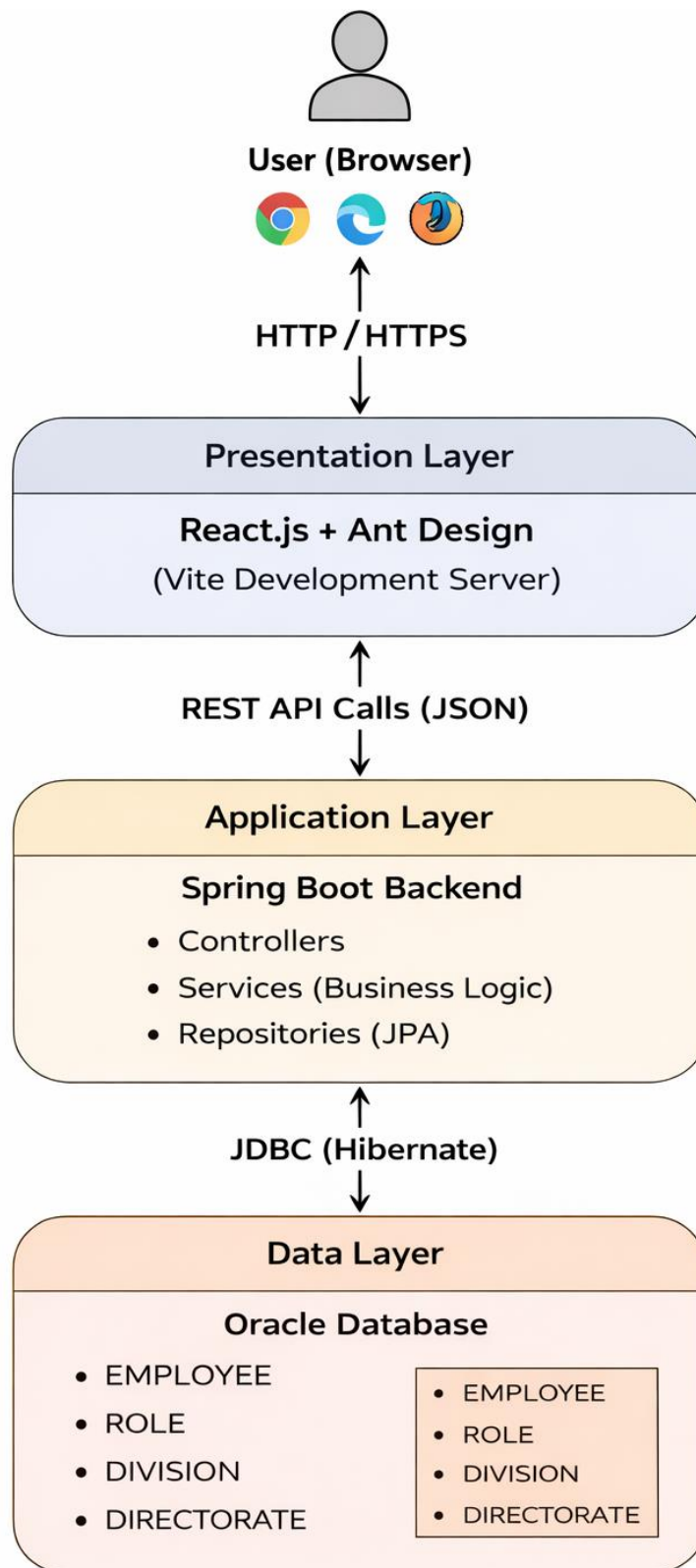
- EMPLOYEE
- ROLE
- DIVISION
- DIRECTORATE

Relationships:

- An Employee belongs to a Role
- An Employee belongs to a Division
- A Division belongs to a Directorate

Oracle ensures:

- Data consistency
- Referential integrity
- Structured relational storage



System Architecture of Employee Management System

Fig 2.1.1 System Architecture

Complete Data Flow in the System

1. User performs an action in browser.
2. React frontend sends REST API request.
3. Spring Boot controller receives request.
4. Service layer processes logic.
5. Repository interacts with Oracle database.
6. Database returns data.
7. Backend sends JSON response.

2.2 MODULE DESCRIPTION

The Employee Management System is divided into multiple functional modules to ensure modularity, maintainability, and efficient system operation. Each module is responsible for a specific set of functionalities within the system.

1. Employee Management Module

This is the core module of the system. It manages all employee-related operations.

Functions:

- Add new employee records
- View employee details
- Update employee information
- Delete employee records
- Display employee list in tabular format

Each employee record contains details such as:

- Employee ID
- Employee Name
- Role

- Division
- Directorate

This module ensures proper storage and retrieval of employee data from the Oracle database using REST APIs.

2. Role Management Module

This module handles employee role classification within the organization.

Functions:

- Create and manage employee roles
- Associate employees with specific roles
- Maintain role-based organizational structure

Roles define the position or designation of an employee within the organization.

3. Division Management Module

The Division module manages departmental classification.

Functions:

- Create and manage divisions
- Associate employees with specific divisions
- Maintain organizational hierarchy

Each employee belongs to a division, and divisions may fall under directorates.

4. Directorate Management Module

This module manages higher-level organizational categorization.

Functions:

- Create and manage directorates
- Associate divisions with directorates
- Maintain structured organizational mapping

This module ensures hierarchical consistency within the organization.

5. REST API Communication Module

This module enables communication between frontend and backend.

Functions:

- Handles HTTP requests (GET, POST, PUT, DELETE)
- Transfers data in JSON format
- Ensures seamless frontend-backend interaction

Axios is used in React to communicate with Spring Boot APIs.

6. Database Management Module

This module handles data storage and retrieval.

Technologies Used:

- Oracle Database
- Spring Data JPA
- Hibernate ORM

Functions:

- Persist employee records
- Maintain relational integrity
- Execute CRUD operations
- Manage table relationships

7. User Interface Module

This module handles user interaction.

Technologies Used:

- React.js
- Ant Design

Functions:

- Display employee data in structured tables

- Provide forms for data input
- Ensure responsive and dynamic UI updates

2.3 IMPLEMENTATION

This section describes the technical implementation of the Employee Management System, including frontend development, backend development, database implementation, and API communication. The system is developed using a full-stack architecture with React.js as the frontend, Spring Boot as the backend, and Oracle SQL as the database.

The implementation ensures efficient communication between system components, secure storage of employee data, and proper handling of organizational hierarchy operations such as employee, role, division, and directorate management.

2.3.1 Frontend Implementation

The frontend of the Employee Management System is developed using React.js with Vite as the build tool and Ant Design as the UI component library. The frontend provides a responsive and interactive interface that allows administrators to manage employee records efficiently.

Multiple React functional components are created to implement different system pages, including Employee List, Add Employee, Update Employee, View Roles, and Dashboard. These components use Ant Design elements such as Forms, Tables, Buttons, Input Fields, Select Dropdowns, and Layout components to create a structured user interface.

React Router DOM is used to manage navigation between different pages of the application. Axios is used to send HTTP requests from the frontend to the backend REST APIs. User input is collected through forms, validated, and sent to the backend for processing.

The frontend dynamically displays employee data in tabular format, allowing administrators to view, edit, and delete employee records.

Example files implemented in frontend include:

- ViewEmployees.jsx – Displays all employees in tabular format
- UpdateEmployee.jsx – Updates employee details
- HomePage.jsx – Provides navigation and system overview

- ViewRoles.jsx – Displays role information
- ConfirmDialog.jsx – Handles delete confirmation popup

The frontend ensures dynamic updates and real-time data rendering without page refresh.

2.3.2 Backend Implementation

The backend of the system is developed using the Spring Boot framework and Java programming language. The backend handles business logic, processes user requests, manages employee operations, and interacts with the Oracle database.

The backend follows a layered architecture consisting of:

- Controller Layer
- Service Layer
- Repository Layer
- Entity Layer

Controller Layer

The Controller Layer handles incoming HTTP requests from the frontend using RESTful APIs. It maps endpoints to service methods and returns responses in JSON format.

Example endpoints include:

- GET /employees
- POST /employees
- PUT /employees/{id}
- DELETE /employees/{id}

Service Layer

The Service Layer contains business logic such as employee creation, updating employee information, validating relationships between roles and divisions, and ensuring data consistency.

Repository Layer

The Repository Layer interacts with the Oracle database using Spring Data JPA and

Hibernate. It performs CRUD operations and manages entity relationships.

Entity Layer

Entity classes represent database tables as Java objects. These classes are annotated using JPA annotations such as `@Entity`, `@Id`, `@GeneratedValue`, and `@ManyToOne` to define table mappings and relationships.

Important backend components include:

- `EmployeeController.java` – Handles employee-related API requests
- `EmployeeService.java` – Contains employee business logic
- `EmployeeRepository.java` – Handles employee database operations
- `RoleRepository.java` – Manages role data
- `DivisionRepository.java` – Manages division data
- `DirectorateRepository.java` – Manages directorate data

The backend ensures structured data validation and proper handling of relational mappings.

2.3.3 Database Implementation

The database is implemented using Oracle SQL to store and manage employee-related information in a structured relational format.

The main database tables used in the system include:

EMPLOYEE table – Stores employee information such as employee ID, name, role ID, division ID, and directorate ID.

ROLE table – Stores role details for employee classification.

DIVISION table – Stores division information.

DIRECTORATE table – Stores directorate information representing higher-level organizational units.

Primary keys and foreign key relationships are used to maintain data integrity. For example:

- Employee table contains foreign keys referencing Role and Division tables
- Division table references Directorate

The backend interacts with the database using Spring Data JPA and Hibernate, which provide object-relational mapping for efficient database access.

2.3.4 API Communication Implementation

Communication between frontend and backend is implemented using RESTful APIs. Axios is used in the frontend to send HTTP requests such as GET, POST, PUT, and DELETE.

The backend exposes REST endpoints using Spring Boot controllers. These endpoints handle employee creation, retrieval, update, deletion, and role/division management.

Data is transferred in JSON format between frontend and backend, ensuring lightweight and efficient communication.

This API-based communication enables seamless integration between system layers.

2.3.5 System Integration

All system components are integrated to ensure proper functionality. The frontend interacts with the backend through REST APIs, and the backend interacts with the Oracle database using JPA and Hibernate.

The complete system is tested to ensure correct operation of all modules, including:

- Employee creation
- Employee update
- Employee deletion
- Role and division mapping
- Data retrieval and display

The implementation ensures scalability, maintainability, and efficient system performance through modular architecture and layered design.

CODE SNIPPETS

```
src > main.jsx > ...
1  import { StrictMode } from 'react';
2  import { createRoot } from 'react-dom/client';
3  import { ConfigProvider } from 'antd';
4  import App from './App.jsx';
5  import './index.css';
6
7  const container = document.getElementById('root');
8  const root = createRoot(container);
9
10 root.render(
11   <StrictMode>
12     <ConfigProvider
13       theme={{
14         token: {
15           colorPrimary: '#667eea',
16           colorSuccess: '#10b981',
17           colorWarning: '#f59e0b',
18           colorError: '#ef4444',
19           colorInfo: '#3b82f6',
20           borderRadius: 12,
21           fontSize: 14,
22           fontFamily: 'system-ui, -apple-system, BlinkMacSystemFont, "Segoe UI", sans-serif',
23         },
24       }}
25     >
26     <App />
27   </ConfigProvider>
28 </StrictMode>
29 );
```

Fig 2.4.1 Frontend Code

```
1 package com.example.demo.controller;
2
3 import com.example.demo.model.Directorate;
4
10
11 @RestController
12 @RequestMapping("/api")
13 @CrossOrigin(origins = "http://localhost:5173") // ← CHANGED FROM 3000 to 5173
14
15
16 public class DirectorateDivisionController {
17
18     private final DirectorateRepository directorateRepo;
19     private final DivisionRepository divisionRepo;
20
21     public DirectorateDivisionController(DirectorateRepository dirRepo,
22                                         DivisionRepository divRepo) {
23         this.directorateRepo = dirRepo;
24         this.divisionRepo = divRepo;
25     }
26
27     @GetMapping("/directorates")
28     public List<Directorate> getAllDirectorates() {
29         return directorateRepo.findAll();
30     }
31
32     @GetMapping("/divisions")
33     public List<Division> getAllDivisions() {
34         return divisionRepo.findAll();
35     }
36 }
```

Fig 2.4.2 Backend Code

2.4 WORKFLOW OF THE SYSTEM

The workflow of the Employee Management System describes the step-by-step process of how administrators interact with the system and how the system processes employee-related operations. The workflow explains the flow of data between the frontend (React), backend (Spring Boot), and database (Oracle).

The system follows a structured process to ensure efficient employee creation, management, updating, and retrieval.

Step 1: User Access to the System

The workflow begins when the administrator accesses the Employee Management System through the web interface using a browser.

The React-based frontend loads the application dashboard, which provides navigation options such as:

- View Employees
- Add Employee
- Update Employee
- View Roles

The user interacts with the system through the user interface developed using React and Ant Design components.

Step 2: Employee Creation

When the administrator selects the option to add a new employee, the system displays a form where details such as:

- Employee Name
- Role
- Division
- Directorate are entered.

After submitting the form:

1. The frontend validates the input data.

2. The frontend sends the employee data to the backend using a POST REST API request.
3. The backend controller receives the request.
4. The service layer processes and validates the data.
5. The repository layer saves the employee record in the Oracle database.

Once the employee is successfully created, the backend sends a success response, and the frontend updates the employee list dynamically.

Step 3: Viewing Employee Records

The administrator can view all employee records by selecting the "View Employees" option.

1. The frontend sends a GET request to the backend.
2. The backend retrieves employee data from the Oracle database using JPA.
3. The data is returned in JSON format.
4. The frontend displays the employee information in a structured table format.

The table shows details such as employee name, role, division, and directorate.

Step 4: Updating Employee Details

If the administrator wants to modify an employee's details, the update option is selected.

1. The system retrieves the existing employee details.
2. The frontend displays the data in an editable form.
3. After modifications, the frontend sends a PUT request to the backend.
4. The backend updates the corresponding record in the Oracle database.
5. A success message is returned to the frontend.

The user interface refreshes automatically to reflect the updated information.

Step 5: Deleting Employee Record

The administrator can delete employee records that are no longer required.

1. The user selects the delete option.

2. A confirmation dialog appears.
3. Upon confirmation, the frontend sends a DELETE request to the backend.
4. The backend removes the employee record from the database.
5. The frontend updates the employee list accordingly.

This ensures clean and updated employee records.

Step 6: Managing Roles, Divisions, and Directorates

The system also allows structured management of organizational hierarchy.

When viewing roles or divisions:

1. The frontend sends a GET request to the backend.
2. The backend retrieves data from respective tables.
3. The data is displayed on the user interface.

This ensures proper mapping between employees, roles, divisions, and directorates.

Step 7: REST API Communication

All communication between frontend and backend is handled using RESTful APIs.

- Axios is used in React to send HTTP requests (GET, POST, PUT, DELETE).
- Spring Boot controllers handle these requests.
- Data is transferred in JSON format.
- The backend interacts with Oracle Database using Spring Data JPA and Hibernate.

This ensures smooth and structured communication between system components.

Step 8: System Response and User Interface Update

After each operation such as create, update, delete, or retrieve:

1. The backend sends a response to the frontend.
2. The frontend updates the user interface dynamically.
3. Success or error messages are displayed accordingly.

This ensures real-time interaction between the administrator and the system.

CHAPTER 3

TESTING AND EVALUATION

Testing and evaluation are critical phases in the development of the Employee Management System to ensure that the system operates correctly, efficiently, and reliably. The purpose of testing is to identify and resolve errors, verify that all system modules function as expected, and confirm that the system satisfies the defined project requirements.

The system was tested using manual testing methods and functional testing techniques. Each module of the system, including employee creation, updating, deletion, viewing, and hierarchical mapping of roles, divisions, and directorates, was tested individually and in integration with other modules. The testing process ensured that the frontend (React), backend (Spring Boot), and database (Oracle) components communicated effectively and produced accurate results.

Functional testing was performed to verify that all system features operate according to the specified requirements. The Add Employee module was tested to ensure that employee details are correctly stored in the Oracle database. The View Employees module was tested to confirm that employee data is displayed accurately in the frontend table. The Update and Delete modules were tested to ensure that employee information can be modified and removed successfully without affecting database integrity.

API testing was performed to verify the communication between the React frontend and Spring Boot backend. All REST API endpoints were tested to ensure proper request handling and accurate responses. Database testing was conducted to verify that employee, role, division, and directorate data are stored, retrieved, updated, and deleted correctly in the Oracle database.

Validation testing was also carried out to ensure that invalid or incomplete input data is handled appropriately and does not cause system failures. The system displays proper success and error messages based on operation outcomes.

The system was evaluated based on performance, accuracy, data consistency, and usability. The results demonstrated that the system performs efficiently under normal operating conditions. The user interface is responsive and intuitive, and all operations such as employee creation, updating, deletion, and retrieval are executed successfully.

Overall, the testing and evaluation process confirmed that the Employee Management System functions correctly and fulfills the intended project objectives.

Testing Types Used

- **Unit Testing**

Each backend component such as controller, service, and repository layers was tested individually to verify correct functionality.

- **Integration Testing**

The integration between frontend, backend, and database was tested to ensure smooth data flow and correct interaction between modules.

- **Manual Testing**

All system features were tested manually through the user interface to verify correct behavior and identify usability issues.

- **API Testing**

REST API endpoints were tested using tools such as Swagger UI or Postman to verify proper request and response handling.

CHAPTER 4

RESULTS

4.1 OUTPUT SNAPSHOTS

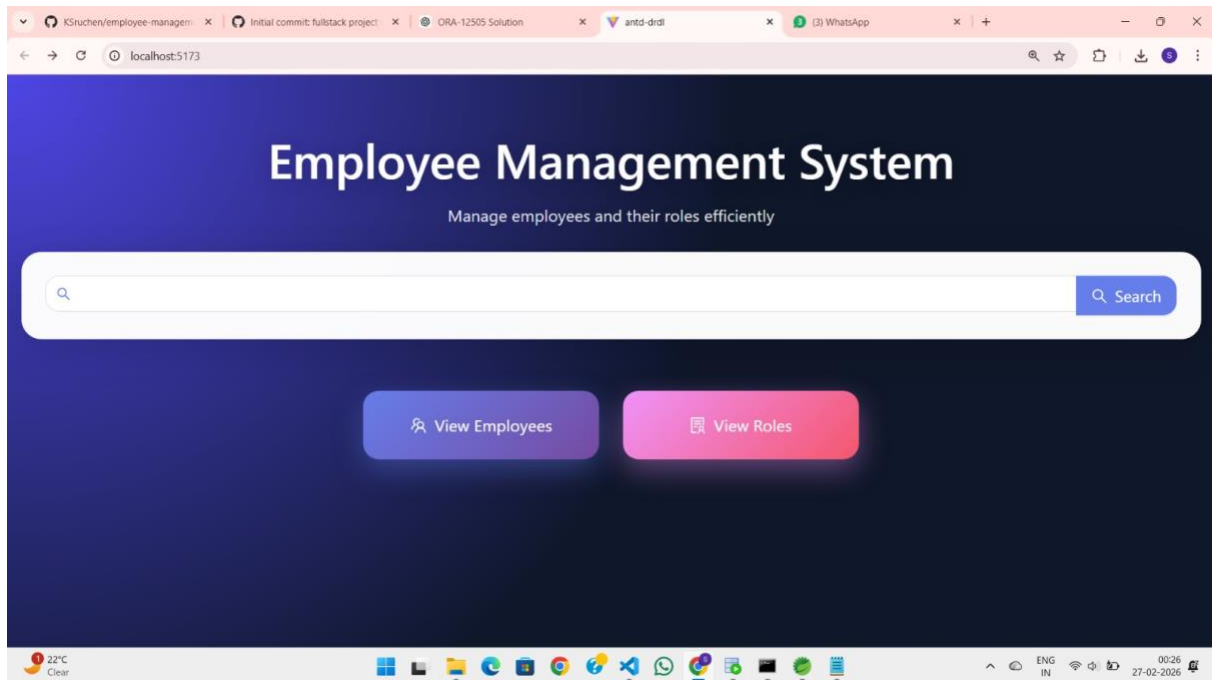


Fig 4.1.1 Dashboard Page

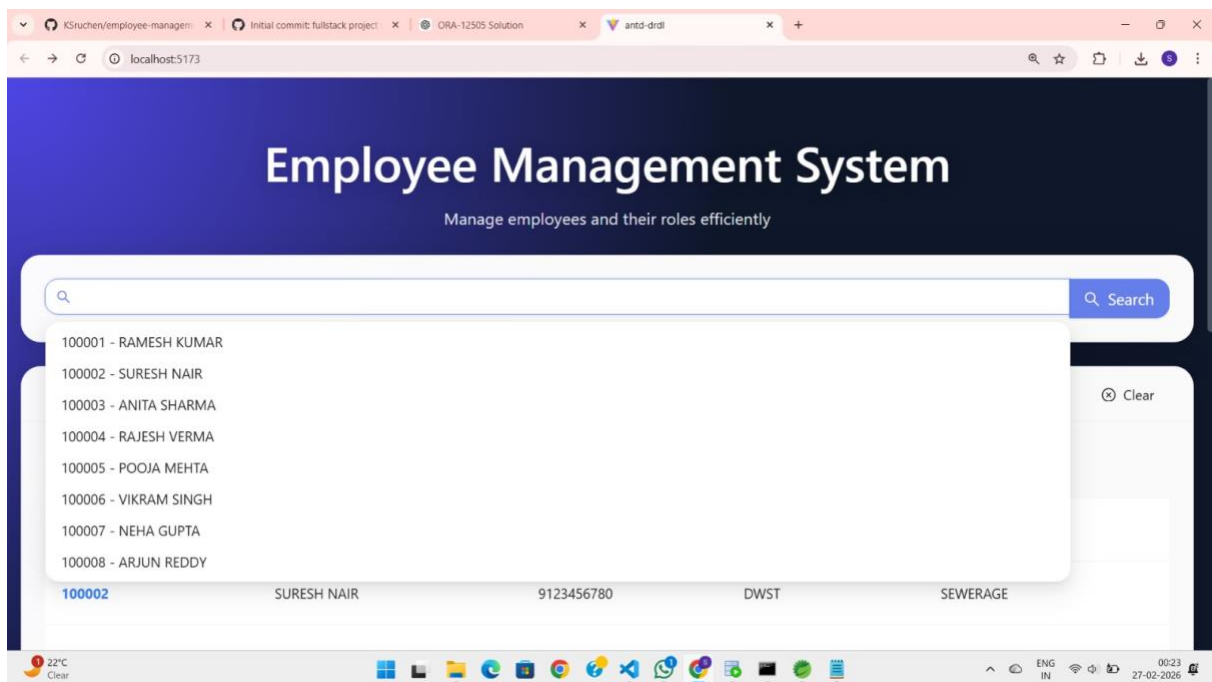


Fig 4.1.2 Search Employee Page

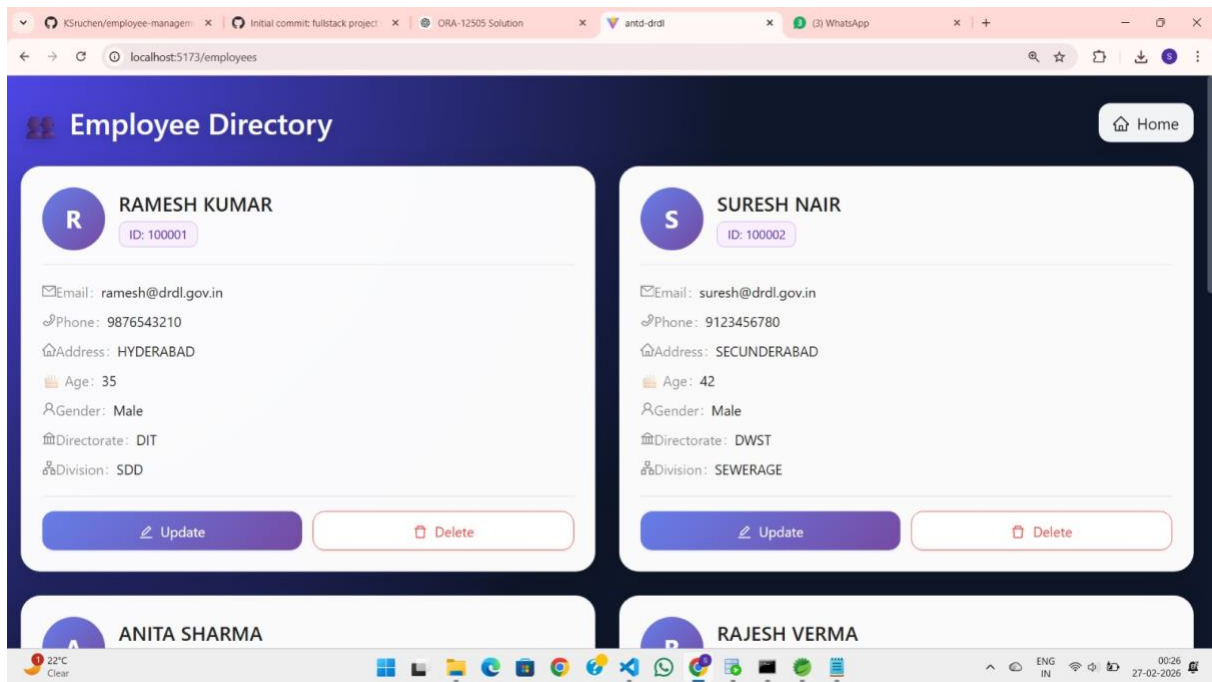


Fig 4.1.3 View Employee Page

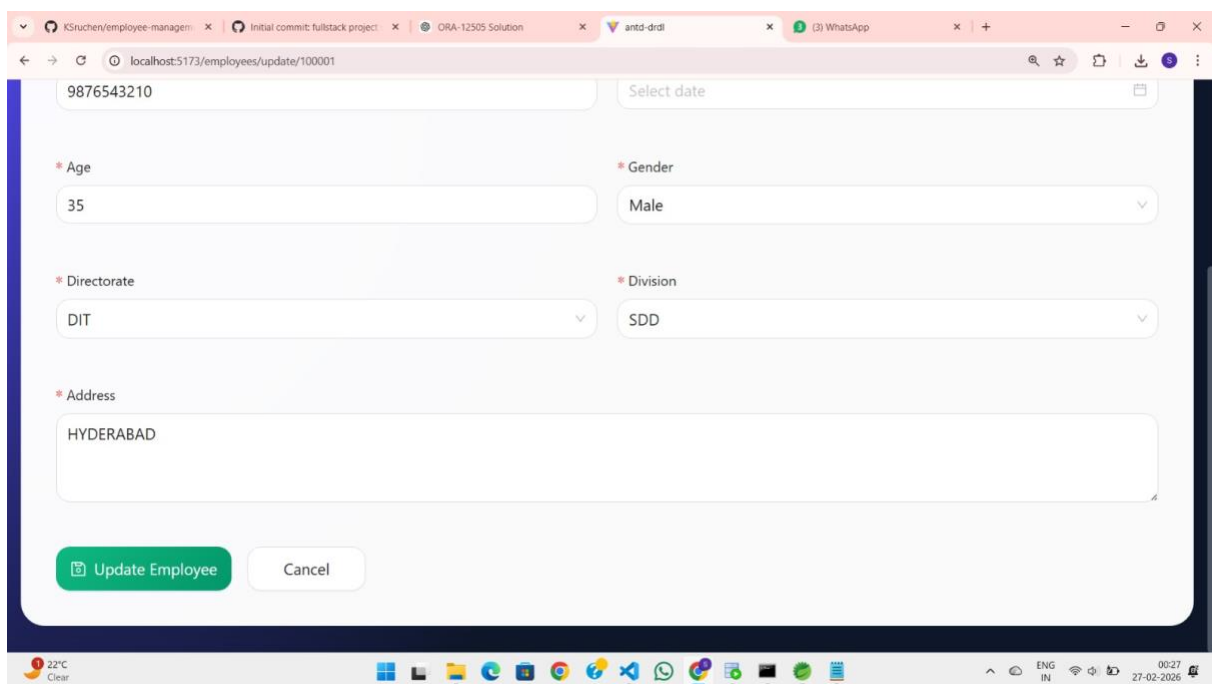


Fig 4.1.4 Update Employee Page

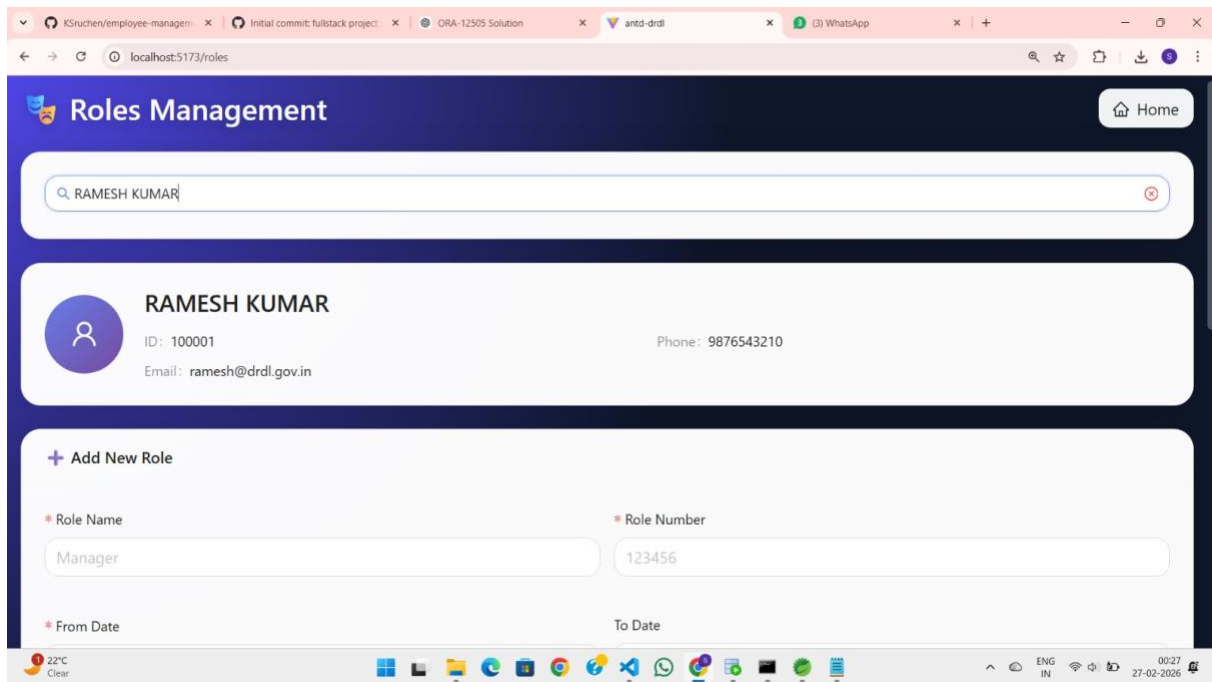


Fig 4.1.5 Role View Page

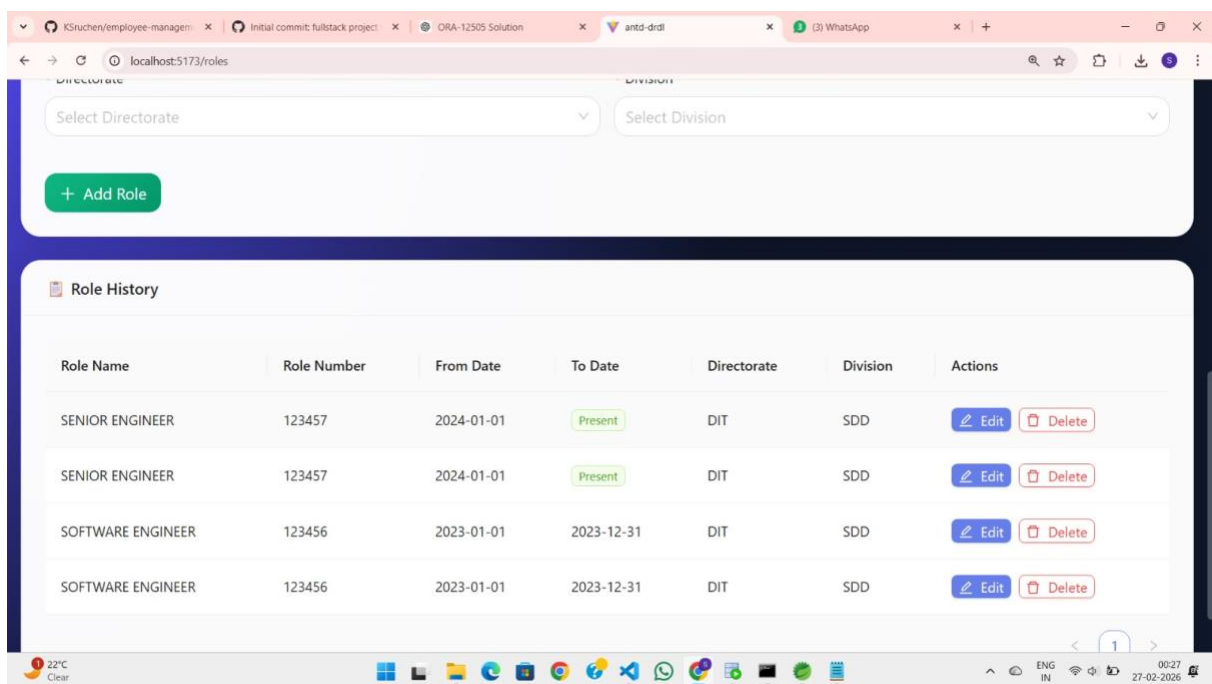


Fig 4.1.6 Manage Roles Page

4.2 CHALLENGES FACED & LEARNING OUTCOMES

Frontend and Backend Integration

One of the major challenges faced during the development of the Employee Management System was establishing seamless communication between the React frontend and the Spring Boot backend. Handling REST API requests using Axios and ensuring proper JSON data mapping between frontend form fields and backend entity objects required careful implementation.

Issues such as incorrect API endpoint mapping, CORS configuration errors, mismatched request/response structures, and improper HTTP methods initially caused data transmission failures. These problems were resolved through systematic debugging, proper configuration of CORS policies in Spring Boot, and consistent API testing using tools like Swagger or Postman.

This challenge improved understanding of RESTful architecture and client-server communication.

Database Connectivity and Entity Relationship Mapping

Integrating Spring Boot with the Oracle database using Spring Data JPA and Hibernate presented challenges related to entity mapping and relational constraints.

Ensuring proper primary key and foreign key relationships between:

- EMPLOYEE
- ROLE
- DIVISION
- DIRECTORATE

required careful schema design. Implementing annotations such as

@ManyToOne, @JoinColumn, and handling cascade types required multiple refinements to ensure referential integrity.

Additionally, handling identity generation strategies, column naming conventions, and resolving constraint violations during insertion and deletion operations required careful database configuration and testing.

This phase strengthened knowledge in relational database design and object-relational mapping.

Handling Hierarchical Relationships

Managing hierarchical relationships between employees, roles, divisions, and directorates was initially complex. Ensuring that employee records were correctly associated with existing roles and divisions required proper validation at both frontend and backend levels.

Special attention was required to prevent deletion of parent records (such as division or role) when dependent employee records existed. Implementing this logic improved understanding of referential integrity and structured data modeling.

Error Handling and Validation

Implementing proper validation mechanisms for employee forms was another challenge. Preventing invalid or incomplete data submission required both frontend form validation and backend-level validation using annotations such as @NotNull, @Size, and other validation constraints.

Handling exceptions gracefully and returning meaningful error messages to the frontend improved overall system reliability.

Testing and Debugging

During testing, several runtime issues such as API response mismatches, incorrect JSON mapping, and database constraint violations were identified.

Debugging required careful inspection of:

- Browser console logs
- Backend server logs
- API request/response data
- Database records

Simultaneous debugging of frontend and backend components enhanced problem-solving skills and improved overall application stability.

OVERALL LEARNING FROM CHALLENGES

The challenges faced during the development of the Employee Management System significantly enhanced understanding of full-stack development principles.

Key learning outcomes include:

- Practical implementation of three-tier architecture
- REST API integration between frontend and backend
- Relational database design and entity mapping
- Debugging and error handling techniques
- Modular and scalable system design

Overcoming these challenges improved technical expertise in React, Spring Boot, and Oracle database integration, and strengthened confidence in developing complete enterprise-level web applications.

CHAPTER 5

CONCLUSION & FUTURE ENHANCEMENT

CONCLUSION

The Employee Management System developed in this project provides a structured and efficient digital solution for managing organizational employee records. The system successfully replaces manual record-keeping methods with a centralized, web-based platform built using modern full-stack technologies.

By integrating **React.js** for the frontend, **Spring Boot** for the backend, and **Oracle Database** for structured data storage, the system demonstrates seamless communication between different layers through RESTful APIs. The implementation of CRUD operations ensures efficient handling of employee information, while the layered architecture enhances maintainability and scalability.

The project effectively manages relationships between employees, roles, divisions, and directorates, maintaining organizational hierarchy and data integrity. The use of Spring Data JPA simplifies database interactions and ensures reliable object-relational mapping between Java entities and database tables.

From a technical perspective, the system illustrates practical implementation of:

- Three-tier architecture
- REST API integration
- Full-stack development principles
- Relational database management
- Modular and scalable design

The developed system improves data accuracy, reduces administrative workload, and provides real-time access to employee information. It serves as a

foundational model that can be further enhanced into a complete Human Resource Management System (HRMS) by incorporating additional modules such as authentication, payroll management, attendance tracking, reporting, and analytics dashboards.

In conclusion, the Employee Management System successfully achieves its objectives by delivering a reliable, scalable, and efficient workforce management solution while demonstrating strong understanding of full-stack application development concepts.

FUTURE ENHANCEMENT

The Employee Management System developed in this project successfully implements core employee record management using a full-stack architecture. The current version provides structured data management, efficient CRUD operations, and seamless frontend-backend integration.

In future versions, the system can be extended with additional supportive features to enhance usability and scalability, such as:

- Implementation of user authentication and role-based access control for improved security.
- Advanced search and filtering options for faster data retrieval.
- Dashboard-based statistical summaries for better organizational insights.
- Pagination support for handling larger datasets efficiently.
- Cloud deployment for improved accessibility and scalability.

These enhancements would further strengthen the system while maintaining its existing modular and scalable architecture.